

Runtime and Cloud Platform — Operator Manual

What this technology does

This layer is the host/runtime substrate underneath the commissioned cloud stack: Ubuntu Server 24.04 LTS, three DigitalOcean Droplets in one private VPC, Docker Engine/Compose, host filesystems, HAProxy/host networking, firewall controls, and the Reserved IPv4 ingress architecture.

It is not a substitute for component-specific HA logic. A healthy Docker daemon does not prove PostgreSQL quorum, ChirpStack health, OpenBao quorum, or Fabric readiness.

Current commissioned host shape

```
ULC-01 / 10.104.0.2
  core HA services
  ChirpStack-1 / Mosquitto-1
  evidence ingest-1 / collector-1
  Fabric adapter-1 ENABLED

ULC-02 / 10.104.0.4
  core HA services
  ChirpStack-2 / Mosquitto-2
  Node-RED standby
  evidence ingest-2 / verifier-1
  Fabric adapter-2 DISABLED

ULC-03 / 10.104.0.8
  core HA services
  Node-RED active
  Grafana
  evidence collector-2 / verifier-2
```

The POC uses three separate hosts because quorum-based services must survive one host loss. All three remain in the same DigitalOcean data center; this is host HA, not region/availability-zone DR.

Public/private network rule

- private east-west service traffic uses the DigitalOcean VPC;
- database, consensus, KMS, evidence object storage, and internal service paths stay on private/loopback listeners as documented;
- the public Reserved IPv4 normal path is commissioned;
- provider-side Reserved IPv4 reassignment/failover acceptance remains a distinct externally managed gate.

Do not open a private service publicly merely to simplify troubleshooting.

Step 1 — Verify host identity and OS

Run on each ULC host:

```
printf 'HOST=%s\n' "$(hostname -s)"
. /etc/os-release
```

```
printf 'OS=%s VERSION=%s CODENAME=%s ARCH=%s\n' "$ID" "$VERSION_ID" "${UBUNTU_CODENAME:-$VERSION_CODENAME}"
"${dpkg --print-architecture}"
ip -br addr
ip route
```

PASS means: the host is the intended Ubuntu 24.04 LTS amd64 member and its private VPC identity matches the expected ULC node.

Do not silently replace one member with a different Ubuntu release or architecture.

Step 2 — Check basic host health before touching containers

```
uptime
free -h
df -hT
df -ih
sudo systemctl --failed
sudo journalctl -p err..alert --since=-30min --no-pager | tail -n 120
```

PASS means: no unexpected failed unit, OOM/disk/inode emergency, or recent host-level critical error explains the application symptom.

A one-off application failure is not a reason to reboot the server.

Step 3 — Check Docker/Compose runtime

```
sudo systemctl is-active docker
sudo docker version
sudo docker compose version
sudo docker info --format 'server={{.ServerVersion}} driver={{.Driver}} cgroup={{.CgroupVersion}} logging={{.LoggingDriver}}'

sudo docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
sudo docker ps -a --filter status=exited --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
```

PASS means: Docker is active, Compose works, intended workloads are running, and there is no unexplained exited/restart-loop container.

Do not prune images, volumes, or build cache until you have identified which artifacts are rollback/recovery assets.

Step 4 — Check listener exposure

```
sudo ss -ltnp
```

Compare listeners with the component manuals. In particular:

- PostgreSQL/etcd/OpenBao/private routing must not become wildcard-public accidentally;
- evidence raw S3 remains loopback-only;
- evidence health ports remain loopback/private;
- only approved public ingress listeners are externally exposed.

If a Docker-published port is unexpectedly public, fix the bind/publish configuration; do not rely only on UFW.

Step 5 — Check firewall state without rewriting it

```
sudo ufw status verbose
sudo nft list ruleset 2>/dev/null | sed -n '1,220p'
```

Current hosts have narrow rules needed for local Docker-bridge-to-host service paths. Preserve the commissioned allowances while investigating.

The DigitalOcean provider Cloud Firewall is externally managed and cannot be proven from a local `ufw` listing alone.

Step 6 — Check private east-west reachability

From each host, test only endpoints relevant to the symptom. For basic VPC reachability:

```
for ip in 10.104.0.2 10.104.0.4 10.104.0.8; do
  printf '%s ' "$ip"
  timeout 2 bash -c "cat < /dev/null > /dev/tcp/$ip/22" 2>/dev/null    && echo TCP22_OK || echo TCP22_FAIL
done
```

Use component-specific protocol checks after TCP reachability. Do not infer service health from ping alone.

Step 7 — Check persistent storage before container recreation

```
sudo find /srv -maxdepth 2 -mindepth 1 -type d -printf '%M %u:%g %p\n' 2>/dev/null | sort | head -n 200

sudo docker inspect $(sudo docker ps -q) --format '{{.Name}} -> {{range .Mounts}}{{.Source}}:
{{.Destination}} {{end}}' 2>/dev/null | sort
```

Before recreating a stateful service, identify its persistent host paths and backup/recovery procedure.

Container recreation is safe only when persistent state and protected configuration are deliberately preserved.

Step 8 — Check runtime resource pressure

```
sudo docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.PIDs}}'
```

Use measured pressure to decide whether a service is undersized. Do not remove HA replicas or security limits simply to make a 2-GiB host look quiet.

Safe package/runtime change procedure

1. Prove the actual failing layer first.
2. Identify the exact package/runtime/service to change.
3. Record current package and image versions.
4. Preserve stateful data and rollback material.
5. Change one host or one service candidate at a time.
6. Keep quorum/active-passive ownership intact.
7. Validate configuration before restart/recreate.

8. Restart only the affected component.
9. Verify the service-specific health check.
10. Verify one representative real operation if the change affects data flow.

Do not perform a three-host package upgrade or Docker restart simultaneously.

Docker image rule

Commissioned workloads should use immutable image digests where the deployment bundle provides them. For an image replacement, record:

```
sudo docker inspect <CONTAINER> --format 'image_id={{.Image}} image_ref={{.Config.Image}}'
```

Do not substitute a mutable `:latest` tag during recovery.

Backup/recovery boundary

The runtime layer must preserve component-specific recovery assets rather than treating `/var/lib/docker` as the backup.

Examples include PostgreSQL recovery material, OpenBao Raft snapshot/recovery material, SeaweedFS raw objects and metadata recovery, Node-RED/Grafana configuration and persistent data, protected PKI/service env files, exact image/release references, and research evidence archives.

Never delete Docker volumes or `/srv` trees as a generic “clean start.”

Host-loss rule

When one ULC host fails:

1. Identify which quorum members/active services were on that host.
2. Verify surviving quorum/leader state.
3. Fence any active/passive writer before promotion.
4. Keep clients on stable logical endpoints.
5. Repair/rebuild the lost host from repository + protected configuration + recovery material.
6. Rejoin/reintroduce one component at a time.

A host rebuild must not require editing every client to a new hard-coded IP.

Troubleshooting map

Symptom	First check
many services fail at once	host resources, Docker, storage, VPC, DNS/time
one container exits	its config/dependencies/logs, not host reboot
private service unreachable	bind/listener -> firewall -> route -> dependency
public path fails but private is healthy	HAProxy/Reserved IPv4/provider boundary

Symptom	First check
disk nearly full	identify owning service/data class before cleanup
OOM/restarts	measured container/host memory and limits
after recreate data vanished	wrong/missing persistent mount; stop further mutation
inconsistent behavior between hosts	image/config/version drift

Completion checklist

- all three hosts identify as the intended Ubuntu 24.04 members;
- no unexpected failed systemd units exist;
- Docker and Compose are healthy;
- no unexplained exited/restarting containers exist;
- disk/inode/memory headroom is acceptable;
- listener exposure matches component manuals;
- private VPC reachability is intact;
- protected persistent paths are known before any recreate;
- provider Reserved-IP failover is not falsely claimed from local host health.

Detailed references

- [../server/cloud-production/01-architecture-decisions-and-scope.md](#)
- [../server/cloud-production/02a-digitalocean-machine-layout-and-specs.md](#)
- [../server/cloud-production/03-digitalocean-vpc-droplets-and-firewalls.md](#)
- [../server/cloud-production/04-host-hardening-dns-pki-and-secrets.md](#)

Supporting runbook: 01-architecture-decisions-and-scope.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

1. HA POC Architecture Decisions

1.1 Goal

Build a **cheap scale model of the future HA deployment**.

We are not sizing for hundreds or thousands of devices. We are proving the relationships between the components using only a few real sensor uplinks.

The acceptance question is:

If any one of ha-01, ha-02, or ha-03 disappears, does the architecture recover without manually editing client IPs, DSNs, or service endpoints?

1.2 POC machine profile

Start with three identical small Droplets using the same lean host OS:

```
OS: Ubuntu Server 24.04 LTS x64
DigitalOcean image slug: ubuntu-24-04-x64

ha-01  1 vCPU / 2 GiB / 50 GiB
ha-02  1 vCPU / 2 GiB / 50 GiB
ha-03  1 vCPU / 2 GiB / 50 GiB
```

This is intentionally aggressive. It is the **minimum full-feature POC starting floor**, not a production recommendation. The POC keeps every documented architecture technology; only capacity, concurrency, cache sizes, worker counts, and traffic are reduced. Because the host baseline is now Ubuntu Server 24.04 LTS, all capacity acceptance must be measured again on that OS; do not reuse memory/headroom measurements from an earlier Debian experiment.

Do not use 512 MiB or 1 GiB nodes as the planned full-feature baseline. Those sizes leave too little credible headroom for PostgreSQL/TimescaleDB, Patroni, etcd, OpenBao, Valkey/Sentinel, routing, ChirpStack/MQTT or Node-RED/Grafana, plus the Fabric workers and failover spikes.

Three machines are still non-negotiable because the architecture needs a majority after one host disappears:

```
3 voters
lose 1
2 remain
2/3 is still a majority
```

1.3 Service placement

```
ha-01
  etcd-1
  PostgreSQL/Patroni-1
  HAProxy
  PgBouncer
  ChirpStack-1
  Mosquitto-1 preferred
  Valkey-1 + Sentinel-1
  OpenBao-1
  Fabric adapter-1

ha-02
  etcd-2
  PostgreSQL/Patroni-2
  HAProxy
  PgBouncer
  ChirpStack-2
  Mosquitto-2 backup
  Valkey-2 + Sentinel-2
  OpenBao-2
  Fabric adapter-2

ha-03
  etcd-3
  PostgreSQL/Patroni-3
  HAProxy private DB + internal MQTT routes
  PgBouncer
  Valkey-3 + Sentinel-3
  OpenBao-3
  Node-RED
  Grafana
```

No dedicated telemetry database server exists in this POC.

1.4 One PostgreSQL cluster, two databases

The POC deliberately reuses the Patroni cluster:

```
PostgreSQL HA cluster
|
+-- chirpstack
|   ChirpStack operational state
|
+-- lorawan_telemetry [TimescaleDB enabled]
    telemetry.uplinks hypertable
    telemetry.measurements hypertable
    telemetry.fabric_outbox ordinary table
```

Why: the test volume is tiny, so a **separate TimescaleDB server** would add another service lifecycle without helping us prove the HA pattern. But TimescaleDB itself is part of the intended deployment, so we keep the feature by installing the extension on every Patroni member and enabling it in `lorawan_telemetry`.

The POC therefore uses the real telemetry shape: TimescaleDB-backed hypertables for sensor time-series data and ordinary PostgreSQL tables for transactional state such as `telemetry.fabric_outbox`. Capacity is reduced; features are not removed.

1.5 Database access decision

Keep PgBouncer because we want the future connection-pooling shape represented in the POC.

Run PgBouncer and a private PostgreSQL HAProxy frontend on all three hosts so local applications can use the same path:

```
application
-> local PgBouncer :6432
-> local HAProxy :15432
-> current Patroni primary :5432
```

Applications use separate database roles even though they share the PostgreSQL cluster.

1.6 PostgreSQL and etcd

Run one Patroni/PostgreSQL member and one etcd member on every Droplet.

Normal state:

```
PostgreSQL: 1 primary + 2 replicas
etcd:       3 voters
```

After one host loss, etcd keeps 2/3 quorum and Patroni can keep or promote a writable PostgreSQL primary.

The POC deliberately uses small PostgreSQL memory settings and PgBouncer so the database does not preallocate production-sized caches or connection counts.

Phase 12A adds a **separate private** Node-RED MQTT frontend on logical `ha-03` / provider host `u1c-03`: `10.104.0.8:18884` -> Mosquitto :8886. This is deliberately distinct from the ChirpStack-specific :18883 -> :8885 routes on `u1c-01/02`. It gives Node-RED a stable Mosquitto-1-preferred/Mosquitto-2-backup mTLS path without pinning it to one app host or exposing `u1c-03` publicly.

1.7 Valkey

Use:

```
3 Valkey data servers
3 Sentinels
Sentinel quorum = 2
```

We are proving primary promotion, not cache scale. Do not add Valkey Cluster sharding.

1.8 MQTT

Use two Mosquitto brokers:

```
ha-01 -> Mosquitto-1 preferred
ha-02 -> Mosquitto-2 backup
```

This proves automatic broker-service failover. It does not pretend that Mosquitto broker state is replicated.

The Raspberry Pi local persistent Mosquitto remains the bounded uplink buffer.

1.9 ChirpStack

Run exactly two instances:

```
ha-01 -> ChirpStack-1
ha-02 -> ChirpStack-2
```

Two is enough to prove Network Server application failover. A third instance would only add load to the POC.

The gateway offline queue is not a substitute for the second ChirpStack instance because it cannot perform OTAA processing or live downlink scheduling.

1.10 OpenBao

Run three OpenBao Integrated Storage/Raft members:

```
OpenBao-1
OpenBao-2
OpenBao-3
quorum = 2
```

For this POC, three members are enough to demonstrate one-node KMS survival. A larger future production KMS can be designed after this proof.

The Fabric adapter never receives an exportable signing private key; it calls OpenBao Transit through the stable private KMS endpoint.

1.11 Fabric adapter and outbox

The **target architecture** supports two fenced workers, but only one is currently enabled:

```
ha-01 / ULC-01 -> Fabric adapter-1  ENABLED production writer
ha-02 / ULC-02 -> Fabric adapter-2  INSTALLED but WRITE-DISABLED standby
```

The reviewed implementation/image and HRC Task 37 handoff are commissioned. The remaining Fabric HA boundary is proving ownership/fencing so ULC-02 cannot create concurrent external side effects with ULC-01. Until that acceptance is deliberate and recorded, keep adapter-2 disabled.

Both read the **same** `telemetry.fabric_outbox` table from `lorawan_telemetry` through the PostgreSQL HA path.

Because that outbox is now in the Patroni cluster, it is no longer tied to `ha-03`.

The flow is asynchronous:

```
Node-RED
-> commit telemetry + selected outbox row
-> adapter claims later
-> OpenBao sign/verify
-> external Fabric Gateway
-> commit result stored back in outbox
```

Fabric/KMS failure must not block an otherwise valid telemetry commit.

1.12 Node-RED and Grafana

Keep one Node-RED and one Grafana on ha-03 because this is a control-plane HA POC, not a full application-platform HA test.

If ha-03 fails:

LoRaWAN control plane	continues
PostgreSQL databases	continue
existing Fabric outbox	remains available; deployed adapter-1/2 may continue existing work
Node-RED ingestion	pauses
Grafana	pauses

That limitation is acceptable for this POC and must be stated clearly.

1.13 Public ingress

Do **not** buy a managed Network Load Balancer for this POC. Reuse HAProxy on ha-01 and ha-02 behind one DigitalOcean Reserved IPv4:

```
chirpstack.<DOMAIN> ----+
                        +--> Reserved IPv4
mqtt.<DOMAIN> -----+
                    |
                    | assigned to one app host
                    v
                ha-01 HAProxy OR ha-02 HAProxy
                    |
                :443 -> ChirpStack
                :8883 -> active Mosquitto
```

The two public HAProxy listeners bind to each Droplet's **anchor IP**, not its VPC IP and not 0.0.0.0. A small failover agent on ha-01/02 probes the public service, confirms the candidate is locally healthy, takes an etcd distributed lock, and uses the DigitalOcean API to reassign the Reserved IP when the current owner fails.

The Reserved IP is active/passive and can belong to only one Droplet at a time. There is no automatic failback: after recovery, the returning host becomes a standby candidate until an operator deliberately moves the address later.

See [10-self-managed-public-ingress.md](#).

1.14 POC scope

This POC **does prove** when the corresponding implementation is present:

```
always executable infrastructure scope:
  one-host quorum survival
  PostgreSQL primary failover
  Valkey primary failover
  public Reserved-IPv4 ownership failover between ha-01/ha-02 with unchanged DNS
  public ChirpStack failover
  MQTT service failover
  OpenBao one-member loss
  shared outbox survives PostgreSQL-primary changes
  no manual client endpoint edits during failover

required full-feature execution gates:
  Fabric worker redundancy -> ULC-01 writer is commissioned; ULC-02 remains fail-closed until
  ownership/fencing + failover acceptance
```

```
gateway-integrity v2    -> reviewed v2 runtime components and real gateway lineage are commissioned;  
retain the evidence lane during counted tests
```

This POC **does not prove**:

```
production capacity  
production RPO/RT0  
large device fleets  
large telemetry retention  
multi-region disaster recovery  
full Node-RED HA  
full Grafana HA  
external Fabric network HA  
production compliance/security sizing  
survival of two simultaneous Droplet failures
```

1.15 Failure tests

Test one failure at a time:

```
ha-01 loss  
ha-02 loss  
ha-03 loss  
PostgreSQL primary process loss  
Valkey primary loss  
Mosquitto-1 loss  
ChirpStack-1 loss  
OpenBao-1 loss  
Fabric adapter-1 loss      # only when adapter execution gate is satisfied  
external Fabric outage     # full reconcile/drain test only when adapter exists  
4G outage and gateway queue drain
```

Restore full redundancy before starting the next failure.

See [19-cloud-ha-grafana-deployment-day-runbook.md](#).

Supporting runbook: 02a-digitalocean-machine-layout-and-specs.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

2A. Absolute-Minimum 3-Server HA POC Layout

This page is the source of truth for the **cheap HA proof of concept**.

The goal is not to imitate production load. The goal is to imitate production **structure**.

Think of it as a small architectural model:

future system: same roles + much more capacity
POC system: same roles + only a few sensor messages

2A.1 Why we still need three Droplets

We can reduce CPU, RAM, storage, and traffic, but we cannot reduce the three **host failure domains** if we want to demonstrate majority-based HA.

3 voters
lose one host
2 voters survive
2/3 majority remains

Use:

Host	OS	Plan	vCPU	RAM	SSD
ha-01	Ubuntu Server 24.04 LTS x64	Basic	1	2 GiB	50 GiB
ha-02	Ubuntu Server 24.04 LTS x64	Basic	1	2 GiB	50 GiB
ha-03	Ubuntu Server 24.04 LTS x64	Basic	1	2 GiB	50 GiB

Use the plain DigitalOcean ubuntu-24-04-x64 image on all three. Do not mix Ubuntu releases, do not substitute a newer LTS just because it is newer, and do not use a Marketplace image for one member.

This is the experiment floor **with all architecture features retained**. The Fabric adapter allowance belongs on ha-01/02 even before its reviewed runtime image is available; missing implementation blocks the final full-feature PASS rather than making the machines smaller on paper.

RAM decision:

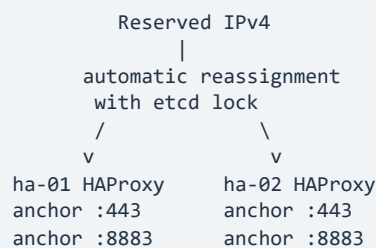
```
512 MiB -> reject
1 GiB  -> reject as the full-feature deployment baseline
2 GiB  -> minimum starting point to test
4 GiB  -> resize target only if measured 2-GiB failover fails
```

If 2 GiB cannot survive the planned failure test on **Ubuntu Server 24.04 LTS** without OOM or persistent swap pressure, resize and keep the evidence. Re-run the full resource/failover evidence after this OS change; Debian measurements do not certify the Ubuntu profile.

These are three independent Droplet/host failures, **not three datacenter or region failure domains**. The self-managed Reserved IPv4 can move only among eligible Droplets in the same DigitalOcean datacenter, so this POC proves one-host loss inside that datacenter. It does not prove datacenter outage, availability-zone independence, or multi-region disaster recovery.

2A.2 One self-managed public entry point

Use one DigitalOcean Reserved IPv4 and reuse HAProxy on ha-01 and ha-02 as the public-ingress candidates:



DNS for both `chirpstack.<DOMAIN>` and `mqtt.<DOMAIN>` points to the Reserved IP. The address is assigned to only one app Droplet at a time. There is **no managed Network Load Balancer** in the minimum POC.

See [10-self-managed-public-ingress.md](#).

2A.3 Exact machine layout

ha-01

```
etcd-1
PostgreSQL / Patroni-1
HAProxy
PgBouncer
ChirpStack-1
Mosquitto-1 preferred
Valkey-1
Sentinel-1
OpenBao-1
Fabric adapter-1
```

ha-02

```
etcd-2
PostgreSQL / Patroni-2
HAProxy
PgBouncer
ChirpStack-2
Mosquitto-2 backup
Valkey-2
Sentinel-2
```

ha-03

etcd-3
PostgreSQL / Patroni-3
HAProxy private database + internal MQTT frontends
PgBouncer
Valkey-3
Sentinel-3
OpenBao-3
Node-RED
Grafana

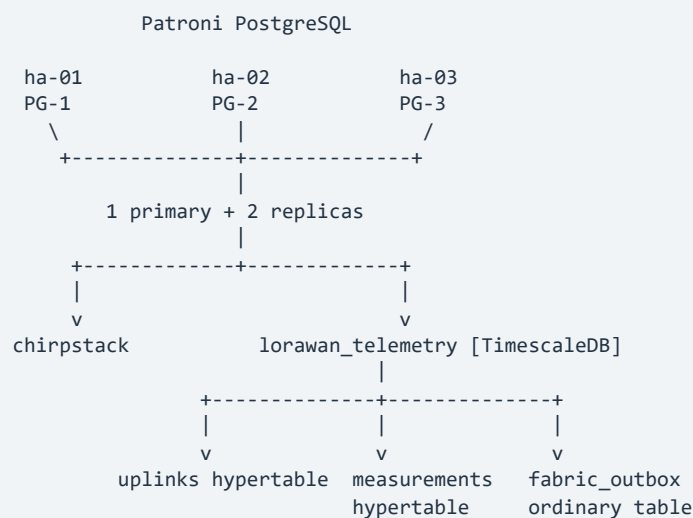
2A.4 What we removed to save money/resources

REMOVE from POC

standalone TimescaleDB container
dedicated telemetry VM
dedicated monitoring VM
managed PostgreSQL
managed Valkey
block volumes
full Prometheus metrics stack
production-size database caches
production-size connection limits

Telemetry still uses TimescaleDB. The difference is that TimescaleDB runs as an extension inside the existing Patroni PostgreSQL members instead of as a separate telemetry server.

2A.5 Database layout



A PostgreSQL primary change therefore moves **both** the ChirpStack and telemetry/outbox write paths together.

2A.6 Database client path

Run PgBouncer and a private HAProxy PostgreSQL frontend on every host:

```
local application
  |
  v
PgBouncer :6432
  |
  v
HAProxy :15432
  |
  v
current Patroni primary :5432
```

Examples:

```
ha-01 ChirpStack-1 -> local PgBouncer
ha-02 ChirpStack-2 -> local PgBouncer
ha-03 Node-RED      -> local PgBouncer
ha-03 Grafana       -> local PgBouncer
ha-03 Node-RED      -> local HAProxy :18884 -> active Mosquitto :8886
ha-01 adapter-1     -> local PgBouncer
ha-02 adapter-2     -> local PgBouncer
```

2A.7 HA groups

```
PostgreSQL / Patroni
  3 members
  1 primary + 2 replicas

etcd
  3 voters
  quorum 2

Valkey
  3 data servers
  1 primary + 2 replicas

Sentinel
  3 voters
  quorum 2

OpenBao
  3 Raft members
  quorum 2

ChirpStack
  2 application instances

Mosquitto
  2 brokers
  preferred + backup

Fabric adapter
  2 workers
  one shared PostgreSQL outbox
```

2A.8 Node-RED active/passive; Grafana remains single

Node-RED is now part of the application availability target. Keep one active instance on ha-03/ulc-03 and a **stopped, pre-staged standby** on ha-02/ulc-02. The standby consumes no steady-state Node-RED container memory while stopped, but the resource test must prove ha-02 can carry the extra Node-RED workload after promotion.

```

normal
-> Node-RED A active on ha-03
-> Node-RED B stopped on ha-02
-> Grafana on ha-03

ha-03 lost
-> fence/confirm old Node-RED unavailable
-> promote Node-RED B on ha-02
-> fresh telemetry processing resumes
-> Grafana may pause

and
-> PostgreSQL stays available
-> existing telemetry/fabric_outbox stay available
-> adapters on ha-01/ha-02 can keep processing eligible jobs
-> ChirpStack/MQTT continue

```

Do not run both Node-RED subscribers concurrently. Grafana remains single for the current POC because dashboard interruption does not stop ingestion. Active/passive Node-RED availability and Grafana availability are separate concerns.

2A.9 Expected one-host failures

Failure	POC expectation
ha-01 lost	when ha-01 owns public ingress, Reserved IPv4 automatically moves to ha-02 with DNS unchanged; ha-02 serves ChirpStack; Mosquitto-2 is available; quorum groups remain 2/3; for a full-feature PASS, deployed adapter-2 must also remain able to claim/reconcile eligible outbox work
ha-02 lost	mirror through ha-01; for this test place the Reserved IPv4 on ha-02 first so automatic public-ingress movement is actually exercised
ha-03 lost	LoRaWAN core stays up; Node-RED/Grafana pause; PostgreSQL/outbox still exist on surviving Patroni members
PG primary lost	Patroni promotes a replica; all DB clients keep the same PgBouncer endpoint
Valkey primary lost	Sentinel promotes; HAProxy follows
OpenBao member lost	2/3 Raft quorum remains
adapter-1 lost	required for full-feature PASS: deployed adapter-2 claims eligible work after valid lease expiry/reclaim; if the reviewed adapter implementation is unavailable, the overall full-feature POC remains BLOCKED

Only one failure is injected at a time.

2A.10 Cost

Planning floor checked 2026-08-20:

```

ha-01  1 vCPU / 2 GiB  $12/month
ha-02  1 vCPU / 2 GiB  $12/month
ha-03  1 vCPU / 2 GiB  $12/month
assigned Reserved IPv4  $0/month
managed Network LB      $0/month (not used)

```


POC floor

\$36/month

No extra cloud resource is added just for telemetry, PgBouncer, OpenBao, Grafana, or the Fabric adapters.

2A.11 When to resize

Do not resize because a production sizing formula says so. Resize only when the POC shows evidence such as:

```
OOM kill
repeated swap thrashing
PostgreSQL cannot complete failover
ChirpStack repeatedly restarts under normal few-sensor load
OpenBao becomes unhealthy because of host memory pressure
one-host failure makes the surviving application node unusable
```

If 2 GiB fails, move to 4 GiB and record that result. The purpose of the POC is to learn the real floor.

2A.12 Build order

Do not maintain a second frozen deployment order in this sizing file. The canonical live order and status are in [00-README.md](#), while [00-build-execution-log.md](#) records what actually happened.

The completed sequence so far is:

```
three active Droplets / host baseline
-> host hardening
-> Docker runtime + 10.104 east-west validation
-> three-member etcd bootstrap and quorum
-> STOP / next technology remains standby
```

The DigitalOcean Cloud Firewall is externally controlled, and Reserved-IP/public-DNS commissioning is not evidenced as a prerequisite that occurred before etcd. Do not rewrite history by placing those provider-side steps into the completed sequence.

When work resumes, activate only the next numbered standby manual, re-check it against the live servers, document the result, then update the canonical sequence/status table.

Continue with [03-digitalocean-vpc-droplets-and-firewalls.md](#) for foundation evidence boundaries.

Supporting runbook: 03-digitalocean-vpc-droplets-and-firewalls.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

3. Create the Minimum DigitalOcean HA Foundation

Evidence boundary: the three active Droplets and their host-observed 10.104.0.0/20 east-west path are evidenced in the live build log. Reserved IPv4 129.212.208.168 and the three DuckDNS public service names are commissioned for the normal path. The current operator still does not independently claim DigitalOcean Cloud Firewall control-panel policy or Reserved-IP reassignment authority; the remaining provider boundary is failover/mobility authorization and acceptance.

This guide creates the **three-Droplet minimum HA test environment** defined in [02a-digitalocean-machine-layout-and-specs.md](#).

Run provider-console/API actions from the administration workstation. Run Linux commands only after SSHing to the named Droplet. Do not paste secrets into this Markdown or a shared terminal transcript.

Before provisioning, fill this non-secret worksheet:

```
DigitalOcean region:
Host OS image/release: Ubuntu Server 24.04 LTS x64
DigitalOcean image slug: ubuntu-24-04-x64
VPC CIDR:
SSH public-key fingerprint:
Administrator source CIDR:
DOMAIN:
chirpstack.<DOMAIN>:
mqtt.<DOMAIN>:
ha-01 planned name:
ha-02 planned name:
ha-03 planned name:
```

Stop here. Do not create resources while the region, VPC CIDR, administrator source, or domain ownership is uncertain. Changing these after quorum services are built creates avoidable rework.

3.1 Foundation state and remaining provider-owned resources

The host-side foundation already evidenced by this build is:

```
3 active Ubuntu 24.04 LTS Droplets
ulc-01 / ha-01
ulc-02 / ha-02
replacement ulc-03 / ha-03
host-observed east-west network on eth1
```

The intended full provider-side target also includes:

```
1 DigitalOcean Reserved IPv4
provider cloud-firewall policy
public DNS records
```

Those provider-side items are **not claimed as completed here**. The DigitalOcean Cloud Firewall is controlled by the account owner/boss, and the current operator must not change it. Reserved-IP and public-DNS commissioning belongs to the later public-ingress phase unless the provider owner supplies evidence that it already exists.

No managed Network Load Balancer, Managed Valkey, Managed PostgreSQL, dedicated role Droplets, or block volumes are required initially.

Why: separating host evidence from provider-control-plane assumptions prevents the manual from reporting a firewall, Reserved IP, DNS record, or provider VPC object that the current operator has not actually verified.

3.2 Droplets

Create:

```
ha-01
  Ubuntu Server 24.04 LTS x64
  image slug: ubuntu-24-04-x64
  Basic 1 vCPU / 2 GiB / 50 GiB

ha-02
  Ubuntu Server 24.04 LTS x64
  image slug: ubuntu-24-04-x64
  Basic 1 vCPU / 2 GiB / 50 GiB

ha-03
  Ubuntu Server 24.04 LTS x64
  image slug: ubuntu-24-04-x64
  Basic 1 vCPU / 2 GiB / 50 GiB
```

Choose **OS -> Ubuntu 24.04 (LTS) x64**, image slug `ubuntu-24-04-x64`, not a Docker/Marketplace/1-Click image. We want the same plain host baseline on every quorum member and will install/pin Docker and the application stack ourselves. Do not select Ubuntu 22.04 or 26.04 for only one node; mixed OS releases create unnecessary package/runtime drift.

Before creating the Droplets, resolve and record the public image metadata from the current DigitalOcean catalog:

```
doctl compute image get ubuntu-24-04-x64 \
  --format ID,Name,Distribution,Slug,Created \
  --no-header
```

Record the numeric image ID and Created value in the worksheet. **Why:** `ubuntu-24-04-x64` is the stable selection name used by these manuals, while the numeric ID proves which provider image build was actually selected for this commissioning run. Do not copy a historical numeric image ID from documentation; resolve it at provisioning time.

Put all three in the same DigitalOcean VPC and region.

Create one host, record its actual size and addresses, then create the next. After all three exist, verify the provider shows exactly the intended three hosts before installing quorum services.

Use tags such as:

```
lorawan-ha
lorawan-test
```

Record the exact Droplet size slugs from the control panel/API at creation time instead of copying a historical slug blindly.

3.3 Operationally validated east-west IP plan

The current active hosts were validated from Linux itself on eth1:

```
ulc-01 / ha-01 10.104.0.2/20
ulc-02 / ha-02 10.104.0.4/20
ulc-03 / ha-03 10.104.0.8/20
```

Cross-node ICMP succeeded and TCP 2380 was proven between hosts before etcd bootstrap. These are therefore the operational HA/east-west addresses used by the current build. The old 10.60.x.x values were design examples only and are removed so they cannot be copied into live service configuration.

The provider control-plane object itself has **not** been independently inspected by the current operator. Do not turn the host-observed result into a claim about DigitalOcean console state. Record any future provider confirmation separately.

Verify the host view with:

```
ip -br address
ip route
```

Stop here if a rebuilt host does not have its documented 10.104.0.x/20 address, the route disappears, or cross-node reachability no longer matches the execution log.

3.4 One Reserved IPv4 for self-managed public ingress

CURRENT NORMAL PATH / PASS: DigitalOcean Reserved IPv4 129.212.208.168 is commissioned and currently assigned to ulc-01 / ha-01, with no managed Network Load Balancer. Public ChirpStack, Evidence, and MQTT normal-path acceptance is recorded in Phase 10. What remains open is provider API authority and controlled reassignment/failover acceptance between ulc-01 and ulc-02; do not infer Cloud Firewall control-panel policy merely from endpoint reachability.

The public shape is:

```
chirpstack.<DOMAIN> ----+
                        +--> Reserved IPv4
mqtt.<DOMAIN> -----+  |
                        | v
                        | ha-01 OR ha-02
                        | anchor-address
                        | HAProxy
                        | /      \
                        | :443    :8883
                        | ChirpStack  MQTT TLS pass-through
```

The address is active/passive and belongs to one Droplet at a time. Automatic movement is implemented by the failover agent on ha-01/02, using an etcd distributed lock plus the DigitalOcean Reserved-IP API.

Do not point DNS at a normal Droplet public IPv4. Both public DNS names point to the Reserved IPv4.

The Reserved-IP failover design is parked in [10-self-managed-public-ingress.md](#). It is **standby** until the application/MQTT paths and provider-side prerequisites are ready; do not execute it during the foundation or etcd phase.

3.5 Target firewall model - provider-owner handoff

The DigitalOcean Cloud Firewall state is currently **unknown to this operator and externally managed**. The rules below describe the intended least-privilege policy for the provider-account owner; they are not instructions for the current operator to modify the cloud firewall.

Public / Reserved-IP-facing

When the provider-owned firewall phase becomes active, allow only what is required:

```
TCP 22    SSH from approved administration sources
TCP 443   public HTTPS to ha-01/ha-02; HAProxy listens only on each host's anchor IP
TCP 8883  public MQTT TLS to ha-01/ha-02; HAProxy listens only on each host's anchor IP
```

If the provider owner enables these rules later, HAProxy must still bind the public service ports to the DigitalOcean **anchor IP**, not 0.0.0.0 and not the normal public Droplet IP. The firewall policy and the application bind policy are separate controls; neither should be assumed from the other.

Do not expose directly:

```
1883 MQTT plaintext
5432 PostgreSQL
6379 Valkey
26379 Valkey Sentinel
2379 etcd client
2380 etcd peer
8008 Patroni REST
8080 ChirpStack backend
8884 Mosquitto private TLS backend
1880 Node-RED
3000 Grafana
8200 OpenBao API private TLS
8201 OpenBao Raft/cluster traffic
18200 HAProxy stable OpenBao KMS frontend
```

Private VPC rules

Allow only between the hosts/services that need them:

```
2379/tcp  etcd client traffic from Patroni/admin nodes
2380/tcp  etcd peer traffic ha-01 <-> ha-02 <-> ha-03
5432/tcp  PostgreSQL replication and HAProxy DB routing
8008/tcp  Patroni health checks
6379/tcp  Valkey data/replication traffic
26379/tcp Sentinel communication
8883/tcp  Reserved-IP public path -> anchor HAProxy MQTT frontend on ha-01/ha-02
8884/tcp  existing gateway-facing HAProxy -> Mosquitto-1/2 private TLS backend
8886/tcp  ulc-03 Node-RED HAProxy -> Mosquitto-1/2 dedicated mTLS backend
8080/tcp  HAProxy -> ChirpStack backends
6432/tcp  local approved DB clients -> PgBouncer frontend
15432/tcp PgBouncer -> local HAProxy PostgreSQL-primary frontend
15433/tcp approved admin/monitor -> HAProxy PostgreSQL-replica test frontend
16379/tcp local app-container -> HAProxy Valkey-primary frontend
18883/tcp local app-container -> HAProxy internal MQTT frontend
8200/tcp  Fabric adapters/approved admins -> OpenBao API backends
8201/tcp  OpenBao-1 <-> OpenBao-2 <-> OpenBao-3 Raft/cluster traffic
```

```
18200/tcp Fabric adapter -> local HAProxy OpenBao KMS frontend
<FABRIC_GATEWAY_PORT>/tcp outbound from Fabric adapter-1/2 to the external Fabric Gateway only
```

Keep the exact allowed source addresses/subnets in the provider-owner firewall record when that information becomes available.

Provider-owner handoff sequence:

1. confirm the actual DigitalOcean VPC/firewall objects and current rules in the provider control plane;
2. preserve the currently working SSH path before any restriction is applied;
3. add only the rules required by services that are actually deployed;
4. verify each permitted path and an unapproved-source rejection;
5. record the result in `00-build-execution-log.md` rather than assuming it succeeded.

The current operator does not perform these provider-side edits.

3.6 Service placement reminder

```
ha-01
  etcd-1
  PostgreSQL/Patroni-1
  HAProxy
  PgBouncer
  ChirpStack-1
  Mosquitto-1
  Valkey-1 + Sentinel-1
  OpenBao-1
  Fabric adapter-1

ha-02
  etcd-2
  PostgreSQL/Patroni-2
  HAProxy
  PgBouncer
  ChirpStack-2
  Mosquitto-2
  Valkey-2 + Sentinel-2
  OpenBao-2
  Fabric adapter-2

ha-03
  etcd-3
  PostgreSQL/Patroni-3
  HAProxy private DB + internal MQTT frontends
  PgBouncer
  Valkey-3 + Sentinel-3
  OpenBao-3
  Node-RED
  Grafana
```

Telemetry and `fabric_outbox` live in `lorawan_telemetry` on the Patroni PostgreSQL cluster. TimescaleDB is enabled there as an extension on the same three PostgreSQL members; no separate TimescaleDB server or public port is added.

3.7 Storage

Use the included SSDs first.

For the test workload, PostgreSQL HA already keeps database copies on three different Droplet root disks. Do not buy three block volumes merely to say storage is separate.

Required safeguards:

```
bounded Docker logs
disk-free monitoring
logical dumps of `chirpstack` and `lorawan_telemetry` copied off the target Droplets before destructive tests
configuration exports
OpenBao recovery/snapshot evidence appropriate to the POC
```

A block volume can be added later if measured data growth or disk pressure requires it.

3.8 Initial host verification

On each Droplet:

```
hostnamectl
cat /etc/os-release
uname -m
uname -r
nproc
free -h
df -h /
ip -br address
ip route
timedatectl
systemctl --failed
```

Expected baseline:

```
all hosts: Ubuntu 24.04 LTS (noble), x86_64/amd64
ha-01: 1 vCPU, about 2 GiB RAM, about 50 GiB root SSD
ha-02: 1 vCPU, about 2 GiB RAM, about 50 GiB root SSD
ha-03: 1 vCPU, about 2 GiB RAM, about 50 GiB root SSD
```

Stop if one host was accidentally created from a different OS image. Rebuild it before quorum bootstrap rather than carrying a mixed host baseline into Patroni/etcd/OpenBao troubleshooting.

Do not begin quorum bootstrap if hostname, private IP, DNS, or time configuration is uncertain.

3.9 DNS and certificates

Public certificate names:

```
chirpstack.<DOMAIN>
mqtt.<DOMAIN>
```

Certificate names must match what clients actually verify. Later execution refined several placeholders from this early infrastructure design. The commissioned identities currently relevant to ChirpStack are:

```
pgbouncer.internal.lorawan.com -> PgBouncer certificates on all three nodes
postgres-ha.internal           -> PostgreSQL member certificates used behind PgBouncer/HAProxy
valkey.internal.lorawan.com    -> every commissioned Valkey server certificate
mqtt.internal.lorawan.com      -> commissioned Mosquitto broker certificates
openbao-kms.internal.lorawan.com -> future OpenBao service identity
```

For containerized ChirpStack, map `pgbouncer.internal.lorawan.com` and `valkey.internal.lorawan.com` to that application host's private VPC IP so the local PgBouncer/HAProxy paths are used. The old `mqtt-ha.internal.<DOMAIN>`:18883 local-per-host target was never commissioned. Phase 9 instead closed ChirpStack MQTT

redundancy with `mqtt.internal.lorawan.com:18883` mapped locally on `ulc-01/02` to `HAProxy :18883 -> Mosquitto :8885`. Phase 12A uses active/passive Node-RED: `ulc-03` is active initially and `ulc-02` is the stopped standby. Each candidate maps `mqtt.internal.lorawan.com:18884` to its own private IP and local `HAProxy`, which routes to the dedicated `Mosquitto :8886` mTLS backends. Phase 10 public MQTT keeps TLS pass-through and therefore reissues broker certificates to retain `mqtt.internal.lorawan.com` while adding the real public `mqtt.<DOMAIN> SAN`. Each `PgBouncer` maps/uses `postgres-ha.internal` for its local `HAProxy` backend. Fabric adapters use the commissioned `openbao-kms.internal.lorawan.com:18200` KMS service boundary; `ULC-01` is the active writer and `ULC-02` remains write-disabled pending HA fencing acceptance.

3.10 Why not two Droplets?

With two etcd voters:

```
normal: 2/2
lose one host: 1/2
majority required: 2
result: no safe quorum
```

With three:

```
normal: 3/3
lose one host: 2/3
majority required: 2
result: quorum survives
```

That is the reason three machines are the absolute minimum for this HA test. It is not an arbitrary architecture preference.

Continue with [04-host-hardening-dns-pki-and-secrets.md](#), then [05-etcd-cluster.md](#).

Supporting runbook: 04-host-hardening-dns-pki-and-secrets.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

4. Host Hardening, DNS, PKI, and Secrets

For the **step-by-step execution record on newly provisioned hosts**, use [04a-host-security-hardening-execution-runbook.md](#). That runbook records what was actually executed, why, the verification result, deviations from the supplied Ubuntu hardening document, and the explicitly excluded yellow-highlighted web-server section.

Current status: the host-security baseline is validated, and the major production PKI/secret boundaries for gateway MQTT, Evidence services, OpenBao, PostgreSQL-facing workloads, and the HRC Fabric adapter have since been commissioned in their component runbooks. Treat generic or future examples in this file as design guidance only; the component-specific current checkpoints and protected host configuration remain authoritative for actual listeners, certificate identities, and secret locations.

4.1 Baseline principles

- Use a supported LTS operating-system image and keep the exact cloud-image identifier for rebuild and rollback. This POC pins **Ubuntu Server 24.04 LTS x64 (Noble)** using the DigitalOcean ubuntu-24-04-x64 image on ha-01/02/03. Do not record only "Ubuntu LTS"; the exact release matters for package repositories and repeatable rebuilds.
- Use named operator accounts, SSH keys, and least privilege.
- Keep database, etcd, MQTT, and Patroni control ports on private interfaces.
- Use a dedicated service account for each daemon where practical.
- Store secrets outside Git and outside shell history.
- Treat transport security per service as a phase-specific control. The **current validated etcd checkpoint uses HTTP only on the private 10.104.0.0/20 east-west network**; do not claim etcd mTLS is deployed. If etcd transport TLS is added later, use a dedicated cluster CA and member/client identities. Gateway-facing MQTT and other future public/private services must be refined and validated when their phases become active.
- Apply updates through tested rolling procedures, not unattended major upgrades.

4.2 Initial host inspection

Run on every new host:

```
hostnamectl
uname -a
cat /etc/os-release
ip -br address
ip route
lsblk -f
```

```
findmnt
ss -lntup
systemctl --failed
journalctl -p warning -b --no-pager
```

Resolve unexpected listeners, failed units, duplicate addresses, or unmounted data volumes before installing the stack.

4.3 Operator access

Create a named operator and install an approved key through the cloud provisioning process. Confirm a second session works before changing SSH.

Back up and validate:

```
sudo cp -pn /etc/ssh/sshd_config /etc/ssh/sshd_config.before-lorawan-cloud
sudoedit /etc/ssh/sshd_config
sudo sshd -t
sudo systemctl reload ssh
```

Typical policy decisions:

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
AllowGroups <APPROVED_SSH_GROUP>
```

Preserve distribution include files and organization policy. Keep provider-console recovery available.

4.4 Time synchronization

Accurate time is required for TLS, logs, database recovery, and LoRaWAN event interpretation.

```
timedatectl status
timedatectl show -p NTPSynchronized --value
chronyc tracking 2>/dev/null || true
chronyc sources -v 2>/dev/null || true
```

Do not start etcd, Patroni, OpenBao, or certificate-sensitive commissioning while NTPSynchronized is no. Ubuntu may use a time-sync implementation other than chrony, which is why the timedatectl synchronization state is the required gate and the chronyc commands are supplemental when that client exists.

Alert on clock offset. Do not manually step time on a database/quorum node without understanding PostgreSQL, etcd lease, OpenBao, and certificate effects.

4.5 Kernel and resource limits

Review the limits below and change only values supported by measured demand. Keep each non-default value with the workload evidence that required it:

```
nofile limits for MQTT, HAProxy, PgBouncer, and PostgreSQL
vm.overcommit_memory policy
TCP listen and connection tracking capacity
filesystem mount options
transparent huge pages policy for PostgreSQL
```

```
swap policy
OOM behavior and systemd resource controls
```

Do not copy generic sysctl bundles. Apply one reviewed setting at a time and compare before/after behavior.

OpenBao and swap

A small host swap file may be useful as an emergency cushion for other processes, but **OpenBao must not be allowed to page secrets into ordinary unencrypted swap**. This is a security boundary, not a performance tweak.

Use the control that matches the OpenBao runtime selected later:

```
native/systemd OpenBao -> verify the service has MemorySwapMax=0
containerized OpenBao  -> configure the equivalent of --memory-swappiness=0
alternative             -> use an explicitly reviewed encrypted-swap design
```

Do not use swap to make an undersized OpenBao node appear healthy. Sustained host swapping still fails the 2-GiB sizing experiment.

4.6 Package and container runtime trust

- Use official distribution repositories or approved vendor repositories.
- Verify repository signing keys and fingerprints through an independent source.
- Pin container images by immutable digest.
- Keep the software bill of materials and vulnerability-scan result with each immutable image digest so the deployed artifact can be traced and replaced.
- Do not mount the container runtime socket into application containers.
- Do not run a container privileged unless the exact capability need is documented.

Install Docker Engine on the plain Ubuntu Server 24.04 LTS hosts

The DigitalOcean OS image is intentionally **not** a Docker Marketplace image, so install Docker Engine before any manual that runs `docker compose`. Docker's current Ubuntu instructions support Ubuntu Noble 24.04 LTS.

First prove that the host is the expected release. Run on **each** Droplet:

```
. /etc/os-release
printf 'ID=%s VERSION_ID=%s CODENAME=%s ARCH=%s\n' \
  "$ID" "$VERSION_ID" "${UBUNTU_CODENAME:-$VERSION_CODENAME}" "$(dpkg --print-architecture)"

test "$ID" = ubuntu
test "$VERSION_ID" = 24.04
test "${UBUNTU_CODENAME:-$VERSION_CODENAME}" = noble
test "$(dpkg --print-architecture)" = amd64
```

Stop here if any test fails. Do not adapt the repository command to a different Ubuntu release on the fly; either rebuild the mismatched Droplet or deliberately revise and revalidate the whole three-host baseline.

Before installing Docker, inspect the package names Docker currently documents as conflicts with Docker Engine from its official repository:

```
dpkg --get-selections \
  docker.io docker-compose docker-compose-v2 docker-doc docker-buildx \
```

```
podman-docker containerd runc \
2>/dev/null || true
```

A fresh POC host should not contain an existing container workload. If any conflicting package is installed, first verify that `/var/lib/docker`, `/var/lib/containerd`, and the running process list do not contain an unknown workload. Only on a confirmed fresh host, remove the conflicts using the current Docker-documented package list:

Build the removal list first so an empty result does not turn into an ambiguous `apt remove` command:

```
CONFLICTING_DOCKER_PACKAGES="$(dpkg --get-selections \
docker.io docker-compose docker-compose-v2 docker-doc docker-buildx \
podman-docker containerd runc 2>/dev/null \
| awk '$2 == "install" {print $1}')"

printf 'conflicting installed packages:\n%s\n' \
"${CONFLICTING_DOCKER_PACKAGES:-<none>}"
```

If the list is non-empty **and this is confirmed to be a fresh host with no workload**, remove exactly that list:

```
if [ -n "$CONFLICTING_DOCKER_PACKAGES" ]; then
    sudo apt remove -y $CONFLICTING_DOCKER_PACKAGES
fi
```

An empty list is normal on a plain image. Do **not** delete `/var/lib/docker` or `/var/lib/containerd` as part of this procedure, and do not run the removal block on a host whose existing container state has not been identified.

Configure Docker's official **Ubuntu** repository:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
apt list --all-versions docker-ce 2>/dev/null | head -n 20
```

Select and record the **approved Docker Engine version string** in the software worksheet. Do not copy a Docker version number from this manual because repository availability changes over time. Then install that reviewed engine version with Docker's current Ubuntu package set:

```
DOCKER_VERSION_STRING='<APPROVED_DOCKER_VERSION_STRING>'

sudo apt install -y \
docker-ce="$DOCKER_VERSION_STRING" \
docker-ce-cli="$DOCKER_VERSION_STRING" \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

Verify before deploying any application container:

```
sudo systemctl is-enabled docker
sudo systemctl is-active docker
sudo docker version
sudo docker compose version
dpkg-query -W \
    docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo docker info --format '{{.ServerVersion}} {{.LoggingDriver}}'
```

Record the Docker Engine, Compose plugin, containerd, package versions, and package source in [02-capacity-cost-and-ip-plan.md](#). Keep all three hosts on the same reviewed package baseline during initial commissioning.

Bound container logs before creating the stack

Current evidence gap: the live execution log proves Docker Engine 29.7.2, overlayfs, systemd cgroups, cgroup v2, and working docker compose use, but it does **not** contain captured output proving the current default Docker logging driver or that `/etc/docker/daemon.json` was changed to `local`. Treat the procedure below as required hardening to verify before the next container phase; do not retroactively mark it PASS.

Docker's default `json-file` driver does not rotate logs unless configured. On these 50-GiB POC disks, unbounded container logs are an avoidable failure mode.

For a new host, use Docker's rotating `local` logging driver as the default:

```
sudo install -d -m 0755 /etc/docker
```

If `/etc/docker/daemon.json` already exists, **stop and merge this setting with the existing reviewed JSON instead of overwriting it**. On a new empty host, create:

```
sudo tee /etc/docker/daemon.json >/dev/null <<'JSON'
{
  "log-driver": "local"
}
JSON

sudo systemctl restart docker
sudo docker info --format '{{.LoggingDriver}}'
```

Expected result:

```
local
```

This default applies to newly created containers. If containers existed before the change, recreate them only through their normal deployment procedure so they inherit the reviewed logging driver.

Docker and firewall boundary

Do not assume a host UFW rule alone protects ports published by Docker. Docker manages its own packet-filter rules. The target architecture requires a provider-side DigitalOcean Cloud Firewall, explicit private/loopback/anchor IP binds, and `ss -lntup` listener verification. **Current provider-firewall state is not verified by this operator and must not be claimed as active.** Until the provider owner confirms it, rely only on controls that are actually evidenced—service bind addresses, SSH hardening, Fail2ban, and listener inspection—and keep provider-firewall completion as an explicit outstanding control.

Spilo maintainers encourage users to build reviewed images from source because public image releases are not regular. Build in CI, scan, sign, publish to an approved registry, and pin the digest.

4.7 PKI hierarchy

The POC is small, but the certificate **roles and names must match the future deployment shape**. Reduce certificate count only when two functions genuinely share the same trust boundary; do not disable TLS or hostname verification to save setup time.

Use separate trust purposes where operationally practical:

```
Public CA
-> chirpstack.<DOMAIN>

Gateway MQTT CA
-> mqtt.<DOMAIN> broker server certificates
-> unique clientAuth certificate per gateway
-> separate ChirpStack and integration client certificates

Gateway Evidence CA or separately constrained PKI purpose
-> evidence.<DOMAIN> HTTPS server certificate
-> unique evidence-upload client identity per gateway
-> no MQTT publish authorization implied by this certificate

Internal etcd CA
-> unique peer/client certificate per etcd member
-> Patroni etcd client certificate

Internal PostgreSQL CA
-> unique server certificate per DB node
-> PgBouncer server certificates
-> optional application client certificates

Internal Valkey CA
-> unique Valkey/Sentinel certificate per node

Internal OpenBao CA
-> unique OpenBao server certificate per node
```

The gateway MQTT certificate Common Name equals the 16-hex Gateway ID when the broker maps certificate identity to ACL username. The evidence-upload identity must also map unambiguously to exactly one Gateway EUI, but it should use a separate key/purpose from MQTT when operationally practical. Reuse one key across MQTT and evidence upload only under an explicit reviewed PKI decision. Do not reuse one private key across gateways or nodes. Server certificates must include the exact DNS names and private IPs used by clients in Subject Alternative Names.

4.7A Certificate/SAN matrix for this POC

This matrix began as a future service-PKI target, but several rows are now commissioned. Use the names below as the current logical-name baseline for services already deployed; later execution records remain authoritative for exact certificate fingerprints, node SANs, and file paths. The current etcd cluster remains the explicit exception: it is validated with HTTP on the private east-west network, so its TLS row applies only if a later transport-hardening change is scheduled.

Service	Where	Name/SAN that clients verify	Why
ChirpStack HTTPS	HAProxy ha-01/02 anchor listeners	chirpstack.<DOMAIN>	Browser/API TLS through the Reserved IPv4 after ownership moves between app hosts
Mosquitto-1	ha-01	mqtt.internal.lorawan.com plus issued node SANs	Commissioned broker identity verified through HAProxy TLS passthrough

Service	Where	Name/SAN that clients verify	Why
Mosquitto-2	ha-02	mqtt.internal.lorawan.com plus issued node SANs	Backup broker presents the same commissioned service identity
PgBouncer	ha-01/02/03	pgbouncer.internal.lorawan.com, node private name/IP	Local database clients use one stable logical name
PostgreSQL	ha-01/02/03	postgres-ha.internal, node private name/IP	PgBouncer/HAProxy can verify any promoted primary
Valkey	ha-01/02/03	valkey.internal.lorawan.com, node private name/IP	HAProxy/ChirpStack verify any Sentinel-promoted primary
OpenBao	ha-01/02/03	openbao-kms.internal.<DOMAIN>, node private name/IP	Either active or standby can serve the stable KMS path
etcd - future TLS hardening only	ha-01/02/03	each member's private DNS/IP	If transport TLS is added later, peer/client mTLS must identify the actual member; current validated etcd is HTTP on 10.104.0.0/20

Client identities:

```
physical gateway MQTT client -> unique cert, CN = Gateway EUI
ChirpStack MQTT client(s)    -> workload identity, never a gateway identity
Node-RED MQTT client         -> read-only application-event identity
Fabric adapter-1/2           -> separate OpenBao workload identities where practical
```

Stop here. Do not continue if a client will connect using a name absent from the server certificate SAN. Fix the certificate or the service name; do not fall back to `sslmode=require`, disabled hostname checks, or `verify none` as the permanent solution.

4.8 Certificate file layout

Example:

```
/etc/lorawan-pki/
public/
etcd/
postgres/
pgbouncer/
mqtt/
valkey/
openbao/
gateway-ca/
gateway-evidence/  # when the evidence-ingest service is deployed
```

Recommended permissions:

```
sudo install -d -m 750 -o root -g <SERVICE_GROUP> /etc/lorawan-pki/<PURPOSE>
sudo install -m 640 -o root -g <SERVICE_GROUP> <CERT_FILE> /etc/lorawan-pki/<PURPOSE>/
sudo install -m 640 -o root -g <SERVICE_GROUP> <KEY_FILE> /etc/lorawan-pki/<PURPOSE>/
```

Some services require a different owner. Verify the effective service user and never loosen a private key to world-readable as a shortcut.

4.9 Secret inventory

At minimum, manage these independently:

- PostgreSQL superuser, replication, rewind, ChirpStack, backup, and monitoring credentials;
- PgBouncer authentication query or auth-file credential;
- etcd client and peer private keys;
- private MQTT broker administration, ChirpStack, Node-RED, and monitoring identities;
- unique Gateway OS MQTT client private keys and certificate lifecycle records;
- when gateway evidence is enabled, unique gateway evidence-upload client identities/private keys and lifecycle records;
- evidence-ingest service credential, protected evidence-store credential, and verifier database/object-store identities;
- ChirpStack token secret and API credentials;
- Valkey password or certificate bundle;
- object-storage access key scoped to the backup bucket **only when the later WAL/object-storage backup profile is enabled**; it is not required for the minimal HA POC;
- DigitalOcean API token used by automation, including the dedicated public-ingress Reserved-IP failover identity on ha-01/02;
- alerting/webhook credentials;
- OpenBao initialization/recovery material and unseal or reviewed auto-unseal material, stored independently from the running nodes;
- separate least-privilege OpenBao workload identities for Fabric adapter-1 and Fabric adapter-2;
- Fabric Gateway CA certificate, MSP ID, channel/chaincode metadata, and a protected client identity for each adapter worker where the Fabric team's policy supports separate identities;
- Fabric adapter database credentials limited to the telemetry/outbox permissions defined by the integration manual.

Keep gateway-evidence verifier credentials intentionally separate from OpenBao Transit sign authorization and the Fabric client private key. The verifier decides whether source lineage is valid; it must not also own the evidence-signing capability. Likewise, do not copy one Fabric client private key or one long-lived OpenBao adapter credential across both adapter hosts unless the external Fabric/OpenBao identity design explicitly requires and approves that sharing; separate workload identity gives clearer revocation and audit evidence.

For every secret, keep its purpose, consumers, protected storage reference, rotation or expiry trigger, and emergency revocation procedure. Do not place the secret value in the inventory.

4.10 Environment files

Use root-owned files outside the repository:

```
sudo install -d -m 750 /etc/lorawan-cloud
sudo install -m 600 /dev/null /etc/lorawan-cloud/<SERVICE>.env
sudoedit /etc/lorawan-cloud/<SERVICE>.env
```

Before using a rendered Compose configuration, remember that `docker compose config` may print expanded secret values. Prefer `docker compose config --quiet` for syntax validation, and redirect any necessary rendered output to a protected temporary file that is removed immediately.

4.11 DNS and certificate validation

```
getent ahosts <SERVICE_FQDN>
openssl s_client -connect <SERVICE_FQDN>:<PORT> -servername <SERVICE_FQDN> -showcerts </dev/null

# Also test evidence.<DOMAIN>:443 separately once that mTLS service exists.
```

Verify chain, hostname, expiry, key usage, and issuer. Automate expiry alerts with enough time for rollback.

4.12 Audit and logging hygiene

- Do not log AppKey, NwkKey, PostgreSQL passwords, private keys, MQTT passwords, API tokens, or complete connection strings.
- Redact authorization headers and database DSNs.
- Restrict journal and log-aggregation access.
- Define retention and legal requirements.
- Test that debug mode is off after commissioning.

4.13 Hardening acceptance

Current server-first security gaps — 2026-08-29

The original host baseline is accepted, but a later full-stack audit found service-security items that are **not** closed merely by the host-hardening PASS:

- certificate-expiry monitoring is required below, but no commissioned checker/timer is recorded;
- Fabric/KMS incident response depends on OpenBao audit evidence, but no commissioned OpenBao audit device is recorded;
- only the Spilo image has a detailed recorded vulnerability/hardening review; the other immutable service images still need scan/provenance coverage or an explicitly accepted gap;
- Docker log/resource/security options are documented consistently for Spilo/OpenBao, but Node-RED, Grafana, ChirpStack, and etcd require a read-only effective-runtime inventory before any hardening recreate is justified;
- etcd private HTTP remains an explicit POC transport-security exception, not production transport-security signoff;
- Historical commissioning began with UFW inactive, but the current hosts have since been hardened with UFW. Preserve the September 3 narrow Docker-bridge allowances required by ChirpStack (172.18.0.0/16 -> each node-local 10.104.0.x on 16379, 6432, 18883) and Node-RED (172.18.0.0/16 -> 10.104.0.8 on 18884, 6432). Re-audit the effective rules and Docker packet-filter interaction before any firewall rewrite; the provider Cloud Firewall state remains a separate externally managed boundary.

The earlier server-first sequence is complete: the gateway/security evidence runtime is commissioned, the OpenBao audit boundary is closed, and ULC-01 has a commissioned least-privilege Fabric writer. Do not repeat those commissioning steps merely because this older hardening guide describes them as future work. Current changes should preserve certificate-expiry monitoring, deterministic exact-byte validation, and container-security inventory; the open Fabric infrastructure boundary is ULC-02 HA fencing/ownership acceptance, while the September 17 research backlog is upstream outbox finalization/claim eligibility.

Complete only when:

- SSH password and root login policy is verified without lockout;

- public listeners match the approved matrix;
- every private key has a named owner and restrictive mode;
- deployed image digests, package versions, and sources can be identified;
- certificate expiry monitoring is active;
- secrets can be rotated without rebuilding the whole environment;
- logs contain no live secret during a sanitized test;
- when gateway evidence is deployed, one unauthorized/unknown gateway client is rejected by the evidence API, one authorized gateway identity maps only to its own Gateway EUI, and the verifier cannot use OpenBao/Fabric signing credentials.

Next validated build step: [05-etcd-cluster.md](#).

Canonical source: [deployment/technology-guides/16-runtime-and-cloud-platform.md](#). For any mutable production-status statement, check the dated baseline and verified live state.